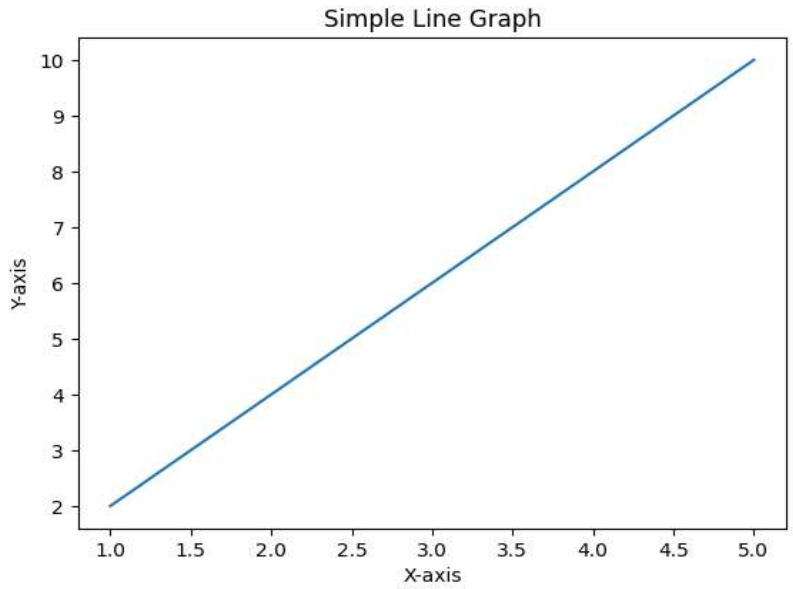
UNIT 1

1.Install Matplotlib and display a simple line graph.

Code:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.plot(x, y)
plt.title("Simple Line Graph")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

Output:



2. Install Scikit-learn and print its version.

Code:

```
import sklearn
print("Scikit-learn version:", sklearn.__version__)
```

Output:

Scikit-learn version: 1.6.1

3. Create a NumPy array containing the first 20 natural numbers.

Code:

```
import numpy as np
arr = np.arange(1, 21)
print(arr)
```

Output:

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
```

4. Create a Pandas DataFrame with student names and grades.

Code:

```
import pandas as pd
data = {'Name': ['Arun', 'Priya', 'Rahul', 'Anu'],
    'Grade': ['A', 'B', 'A+', 'B+']}
df = pd.DataFrame(data)
print(df)
```

Output:

```
...      Name Grade
    0   Arun     A
    1  Priya     B
    2  Rahul    A+
    3    Anu    B+
```

5. Verify the installed versions of all AI libraries on your system.

Code:

```
import numpy
import pandas
import matplotlib
import sklearn
import torch
import transformers
```

```
print("NumPy:", numpy.__version__)
print("Pandas:", pandas.__version__)
print("Matplotlib:", matplotlib.__version__)
print("Scikit-learn:", sklearn.__version__)
print("PyTorch:", torch.__version__)
print("Transformers:", transformers.__version__)
```

Output:

```
NumPy: 2.1.3
Pandas: 2.2.3
Matplotlib: 3.10.0
Scikit-learn: 1.6.1
PyTorch: 2.11.0+cpu
Transformers: 5.20.2
```

6. Create tensors using PyTorch and perform basic mathematical operations.

Code:

```
import torch
a = torch.tensor([2, 4, 6])
b = torch.tensor([1, 2, 3])
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
```

Output:

```
Addition: tensor([3, 6, 9])
Subtraction: tensor([1, 2, 3])
Multiplication: tensor([ 2,  8, 18])
Division: tensor([2., 2., 2.])
```

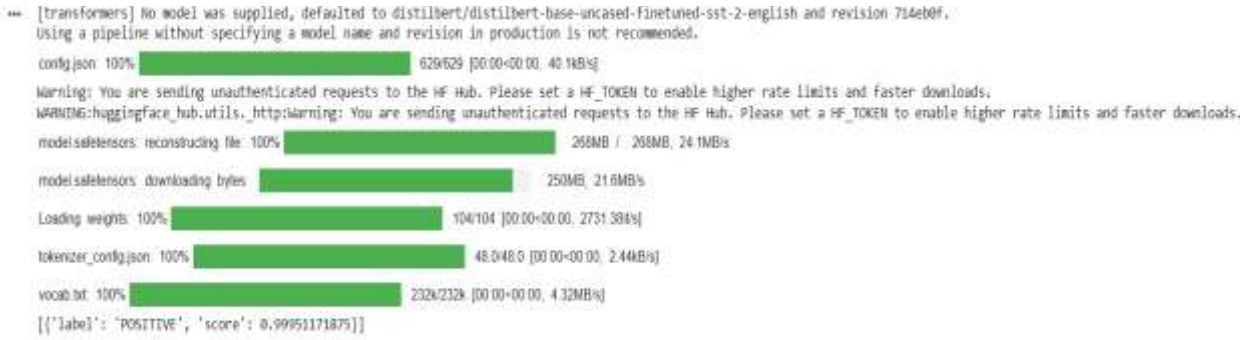
7. Load a Pre-trained Transformer Model using Hugging Face and perform sentiment analysis.

Code:

```
from transformers import pipeline
```

```
sentiment = pipeline("sentiment-analysis")
result = sentiment("I love learning Artificial Intelligence.")
print(result)
```

Output:

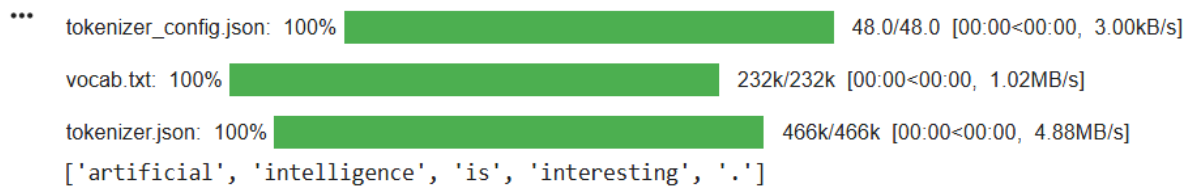


8. Tokenization using BERT Tokenizer

Code:

```
from transformers import BertTokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
text = "Artificial Intelligence is interesting."
tokens = tokenizer.tokenize(text)
print(tokens)
```

Output:



9. Generate text using a pre-trained GPT-2 model.

Code:

```
from transformers import pipeline
generator = pipeline("text-generation", model="gpt2")
result = generator(
    "Artificial Intelligence",
    max_new_tokens=30,
   do_sample=False)
```

print(result[0]["generated_text"])

Output:



10. Load a pre-trained transformer model using the Hugging Face pipeline() API and perform sentiment analysis.

Code:

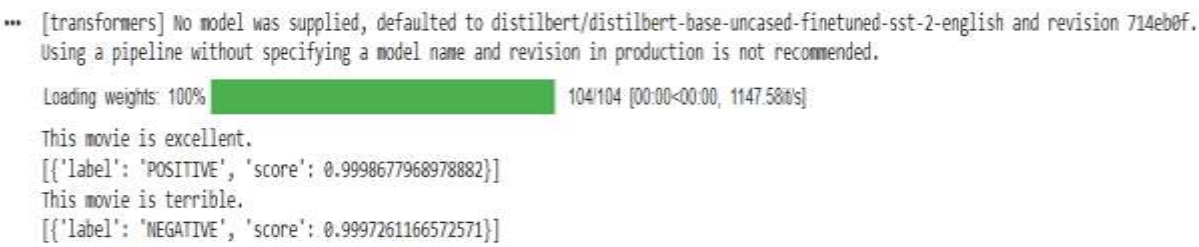
```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

texts = [
    "This movie is excellent.",
    "This movie is terrible."
]

for text in texts:
    print(text)
    print(classifier(text))
```

Output:



11. Load the BERT tokenizer and tokenize a sentence.

Code:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

```
sentence = "Machine learning is powerful."
print(tokenizer.tokenize(sentence))
```

Output:

```
config.json: 100% 570/570 [00:00<00:00, 11.3kB/s]
['machine', 'learning', 'is', 'powerful', '.']
```

12. Load a pre-trained BERT model and generate contextual embeddings.

Code:

```
import torch
from transformers import AutoTokenizer, AutoModel
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")
text = "AI makes computers intelligent."
inputs = tokenizer(text, return_tensors="pt")
with torch.no_grad():
    outputs = model(**inputs)
print("Embedding shape:", outputs.last_hidden_state.shape)
print(outputs.last_hidden_state)
```

Output:

```
model.safetensors: reconstructing file: 100% 440MB / 440MB, 30.4MB/s
model.safetensors: downloading bytes: 415MB, 31.4MB/s
Loading weights: 100% 199/199 [00:00<00:00, 3267.97it/s]
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key                                        | Status     |  |
-------------------------------------------+------------+--+-
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.bias                  | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |

Notes:
- UNEXPECTED:    can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Embedding shape: torch.Size([1, 7, 768])
tensor([[[-0.2289,  0.3845, -0.0841,  ..., -0.4917,  0.1091,  0.5127],
         [ 0.2016,  0.4349, -0.2636,  ..., -0.4460,  0.4402,  0.2851],
         [ 0.0730,  0.6691,  0.2197,  ..., -0.4078, -0.2088,  0.3723],
         ...,
         [ 0.2207,  0.6172, -0.1302,  ..., -0.6000, -0.1205, -0.0511],
         [-0.1529, -0.7385, -0.4472,  ...,  0.2488,  0.4327, -0.2745],
         [ 0.8363,  0.1481, -0.3532,  ...,  0.0693, -0.6202, -0.1382]]])
```

13. Generate text using a pre-trained GPT-2 model.

Code:

```
from transformers import GPT2Tokenizer, GPT2LMHeadModel
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")
prompt = "The future of AI"
inputs = tokenizer(prompt, return_tensors="pt")
outputs = model.generate(
    **inputs,
  max_new_tokens=25,
    do_sample=False,
    pad_token_id=tokenizer.eos_token_id
)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

Output:

```
Loading weights: 100% ████████████████████ 148/148 [00:00<00:00, 1815.04it/s]
The future of AI is uncertain. The future of AI is uncertain.

The future of AI is uncertain. The future of AI is uncertain
```

14. Question Answering using a Pre-trained Transformer

Code:

```
from transformers import pipeline
qa = pipeline(
    "question-answering",
    model="distilbert/distilbert-base-cased-distilled-squad")
context = "Python is a popular programming language created by Guido van Rossum."
question = "Who created Python?"
result = qa(question=question, context=context)
print(result["answer"])
```

Output:

Guido van Rossum

15. Tokenize a sentence using the BERT tokenizer and display the generated tokens and token IDs.

Code:

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

sentence = "Deep learning is amazing."

tokens = tokenizer.tokenize(sentence)

ids = tokenizer.convert_tokens_to_ids(tokens)

print("Tokens:", tokens)

print("Token IDs:", ids)
```

Output:

```
Tokens: ['deep', 'learning', 'is', 'amazing', '.']
Token IDs: [2784, 4083, 2003, 6429, 1012]
```

16. Compare the tokenization outputs of BERT and GPT-2.

Code:

```
from transformers import AutoTokenizer

bert = AutoTokenizer.from_pretrained("bert-base-uncased")

gpt2 = AutoTokenizer.from_pretrained("gpt2")

text = "Learning AI is wonderful!"

print("BERT tokens:", bert.tokenize(text))

print("GPT-2 tokens:", gpt2.tokenize(text))
```

Output:

```
BERT tokens: ['learning', 'ai', 'is', 'wonderful', '!']
GPT-2 tokens: ['Learning', 'ĠAI', 'Ġis', 'Ġwonderful', '!']
```

17. Generate contextual embeddings for a sentence using BERT.

Code:

```
import torch

from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

model = AutoModel.from_pretrained("bert-base-uncased")
```

```
text = "Natural language processing is useful."
inputs = tokenizer(text, return_tensors="pt")
with torch.no_grad():
    outputs = model(**inputs)
embedding = outputs.last_hidden_state
print("Contextual embedding shape:", embedding.shape)
```

Output:



18. Compare contextual embeddings generated for two semantically similar sentences.

Code:

```
import torch
from transformers import AutoTokenizer, AutoModel
from torch.nn.functional import cosine_similarity
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")
sentences = [
    "I like learning AI.",
    "I enjoy studying artificial intelligence."
]
inputs = tokenizer(sentences, padding=True, truncation=True, return_tensors="pt")
with torch.no_grad():
    outputs = model(**inputs)
embeddings = outputs.last_hidden_state[:, 0, :]
similarity = cosine_similarity(embeddings[0], embeddings[1], dim=0)
```

print("Cosine similarity:", similarity.item())

Output:



19. Generate text from a prompt using the GPT-2 language model.

Code:

```
from transformers import pipeline

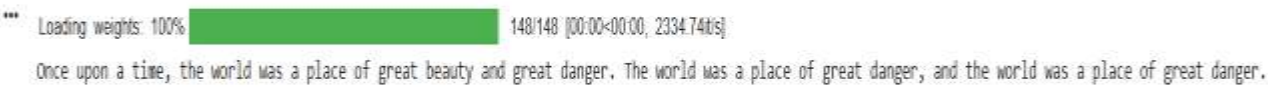
generator = pipeline("text-generation", model="gpt2")

prompt = "Once upon a time"

result = generator(
    prompt,
    max_new_tokens=40,
    num_return_sequences=1,
    do_sample=False
)

print(result[0]["generated_text"])
```

Output:



20. Generate and compare responses for multiple prompts using a pre-trained LLM.

Code:

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompts = [
    "Artificial Intelligence is",
```

```
    "Climate change can be",
    "The importance of education is"
]
for prompt in prompts:
    result = generator(
        prompt,
        max_new_tokens=25,
        do_sample=False
    )
    print("\nPrompt:", prompt)
    print("Response:", result[0]["generated_text"])
```

Output:

